


# JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY VIZIANAGARAM

## VIZIANAGARAM – 535003

## ANDHRA PRADESH

# ELECTRICITY USAGE TRACKER

**By**

**A.MAHESWARULU**

(Regd. No. 24VV1A1204)

Under the supervision of

**MRS. MADHUMITA CHANDA**

Department of Information Technology

JNTU-GV, Vizianagaram

Academic Year: 2026–2027

# CONTENTS

1. Introduction ..... 3
2. Project Overview and Requirements ..... 4
3. Project Architecture and Structure ..... 6
4. Flutter Development Environment Setup ..... 8
5. Dart Programming Fundamentals Used in the Project ..... 11
6. Creating and Configuring the Flutter Project ..... 13
7. Electricity Usage Tracker – Case Study ..... 14
8. User Authentication and Profile Management ..... 16
9. Meter Reading Management ..... 18
10. Consumption and Bill Estimation ..... 20
11. Usage Analytics and History ..... 22
12. Alerts and Energy-Saving Guidance ..... 24
13. Exploring Flutter Widgets in the Project ..... 26
14. Responsive UI Design ..... 28
15. Navigation Between Screens ..... 30
16. Stateful and Stateless Widgets ..... 31
17. State Management ..... 32
18. Custom Widgets and Styling ..... 33
19. Forms and Validation ..... 35
20. Local Data Storage with SQLite ..... 37
21. Database and Data Models ..... 39
22. Animations and User Experience ..... 41
23. Testing and Debugging ..... 42
24. Application Workflow ..... 44
25. Security and Data Validation ..... 49
26. Conclusion ..... 50

# 1. Introduction

Flutter is an open-source UI framework developed by Google for building cross-platform applications from a single codebase. Flutter uses the Dart programming language and provides a widget-based approach for creating interactive mobile interfaces.

The Electricity Usage Tracker is a Flutter application that helps a user record meter readings, calculate units consumed and estimated bills, review usage history, and understand consumption patterns. The application stores its data locally and presents the information through a responsive interface.

## Advantages of Flutter

- Cross-platform development from a common codebase.
- Hot reload for faster interface development.
- A large collection of customizable Material widgets.

## Dart Language

Dart is object-oriented, strongly typed, and supports asynchronous programming through Future, async, and await. These features are used while reading and writing SQLite data and while updating screens after user actions.

## Key Flutter Features Used in This Project

- Widget-based screens for dashboards, forms, cards, charts, and lists.
- Stateful widgets for readings, theme selection, and changing usage data.
- Navigation between authentication, dashboard, history, analytics, and profile screens.

# 2. Project Overview and Requirements

The Electricity Usage Tracker is a mobile application for maintaining electricity meter records. A user can register, sign in, add readings, estimate a bill, review past consumption, inspect monthly charts, and configure a tariff value used in calculations.

## Problem Statement

Electricity consumption is often recorded manually, which makes it difficult to compare monthly usage or estimate a bill before it arrives. Users need a simple way to preserve meter readings, calculate units and charges consistently, and identify high-consumption periods.

## Main Objectives

- Provide account registration, login, and profile management.
- Allow users to add, view, and delete meter readings.
- Calculate consumed units from consecutive readings.
- Estimate electricity bills using a configurable tariff.
- Present usage history and monthly analytics.
- Provide in-app high-usage alerts and energy-saving guidance.
- Persist user data and readings locally using SQLite.
- Support light and dark themes with a responsive interface.

## Application Scope

The application is a single-device tracker. It does not connect to an electricity provid
er, read a smart
meter, synchronise data to a cloud service, or send scheduled notifications outside the
application.

# 2. Project Overview and Requirements

## Functional Requirements

| Module | Requirement |
|---|---|
| Authentication | Registration, login, logout, duplicate-email checks, and form validation. |
| Profile | View and update the user name, email address, and electricity tariff. |
| Readings | Add, list, and delete meter readings for the signed-in user. |
| Calculation | Compute units from consecutive readings and calculate the bill from the tariff. |
| Bill Estimate | Estimate a bill from a user-entered number of units without saving a reading. |
| Analytics | Display monthly usage and bill charts, averages, and a highest-usage insight. |
| Alerts | Show high-usage warnings, monthly reminders, and energy-saving tips. |
| Storage | Persist users, readings, and settings locally with SQLite. |

## Non-Functional Requirements

- Usability: clear forms, readable cards, and straightforward navigation.
- Performance: database work is asynchronous and does not block the interface.
- Maintainability: screens, models, services, and theme code are separated.
- Scalability: additional modules can be added without rebuilding every screen.
- Portability: the interface adapts to different mobile screen sizes.

# 3. Project Architecture and Structure

The application follows a modular Flutter structure. Interface code is separated from models, database access, and shared theme configuration so that each part can be maintained independently.

## Project Directory

```
electricity_usage_tracker/
├── lib/
│   ├── main.dart
│   ├── database/
│   │   └── database_helper.dart
│   ├── models/
│   │   └── electricity_reading.dart
│   ├── services/
│   │   └── database_service.dart
│   ├── screens/
│   │   ├── splash_screen.dart
│   │   ├── landing_screen.dart
│   │   ├── login_screen.dart
│   │   ├── register_screen.dart
│   │   ├── dashboard_screen.dart
│   │   ├── add_reading_screen.dart
│   │   ├── bill_estimate_screen.dar
t
│   │   ├── usage_history_screen.dar
t
│   │   ├── analytics_screen.dart
│   │   ├── alerts_screen.dart
│   │   ├── profile_screen.dart
│   │   └── personal_information_scr
een.dart
│   └── theme/
│       ├── app_theme.dart
│       └── theme_controller.dart
├── android/
├── assets/
├── test/
└── pubspec.yaml
```

# 3. Project Architecture and Structure

## Description of Project Modules

| Directory | Purpose |
|---|---|
| lib/ | Contains the Dart source code for the application. |
| database/ | Defines the local SQLite connection and table creation logic. |
| models/ | Represents an electricity reading and converts database rows into objects. |
| services/ | Provides asynchronous CRUD operations and tariff settings. |
| screens/ | Contains complete page-level interfaces and user interactions. |
| theme/ | Defines the light and dark visual styles and persisted theme selection. |
| assets/ | Stores static application resources declared in pubspec.yaml. |
| test/ | Contains automated tests; the current file is the default Flutter template test. |
| pubspec.yaml | Declares Flutter dependencies, assets, and project configuration. |

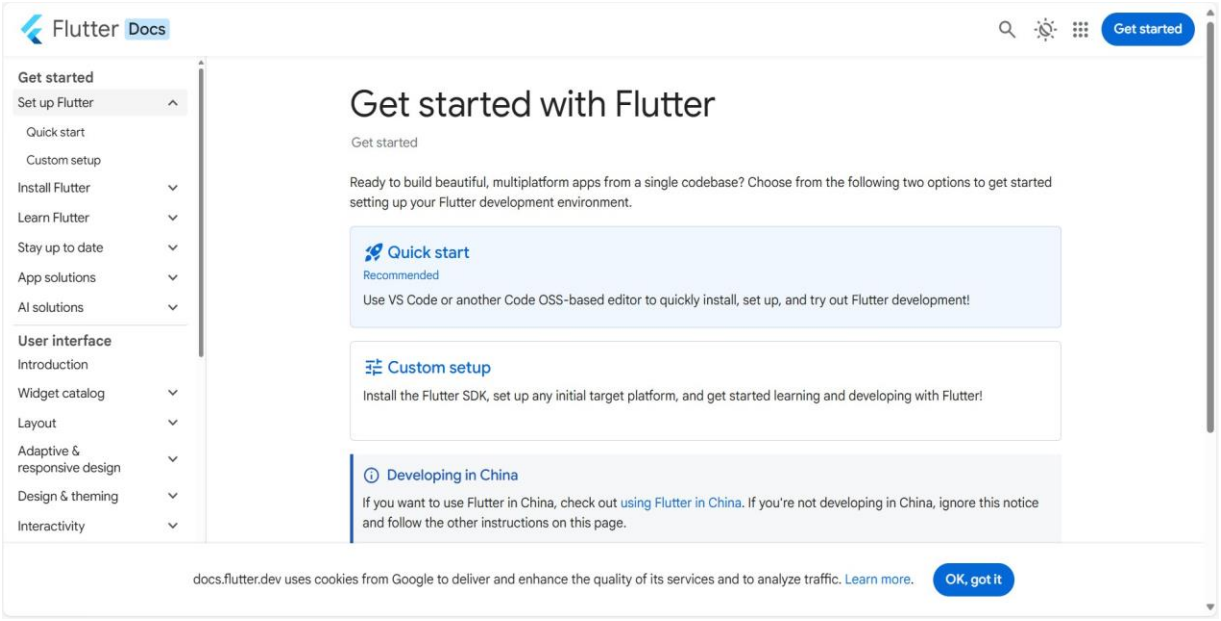
## Architecture Flow

```
Flutter Widgets and Screens
        ↓
Local UI State and ThemeCont
roller
        ↓
DatabaseHelper / DatabaseSer
vice
        ↓
sqflite SQLite Database
        ↓
users, readings and settings
 tables
```

# 4. Flutter Development Environment Setup

## Flutter SDK Installation

1. Download the stable Flutter SDK from the official Flutter website.
2. Extract the SDK to a location such as C:\src\flutter.
3. Add C:\src\flutter\bin to the Windows PATH.
4. Open a new terminal and run flutter --version.
5. Run flutter doctor to identify missing components.




Flutter SDK installation and toolchain verification

## Android Toolchain Without Android Studio

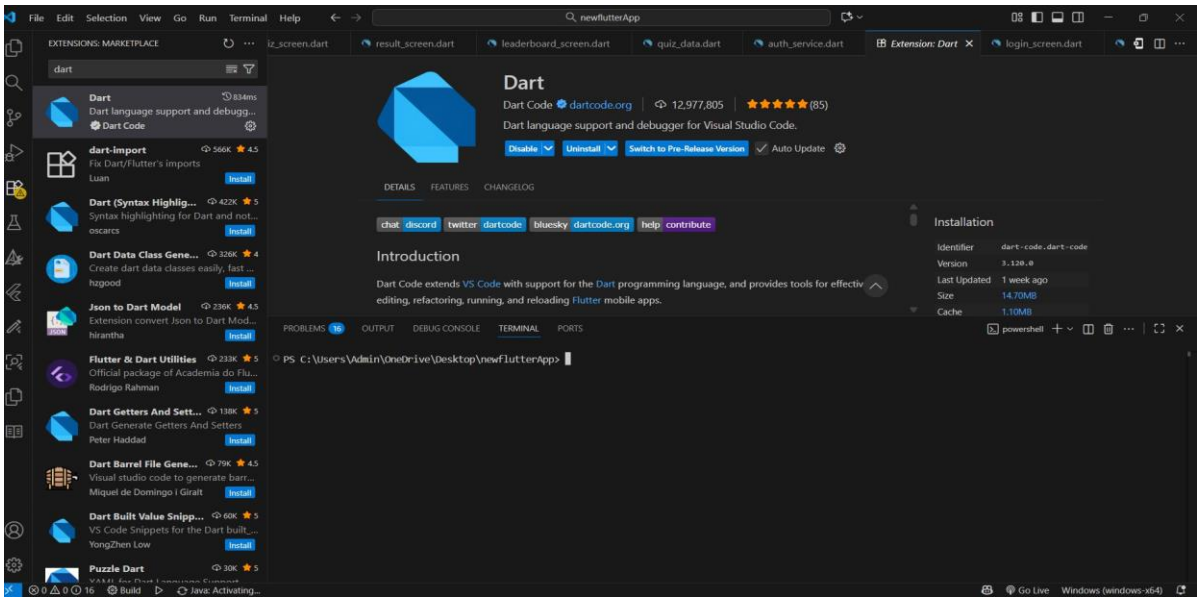
The project can be developed with Visual Studio Code and the Android command-line tools. Android Studio is not required as the code editor.

1. Install the Android SDK command-line tools.
2. Configure the Android SDK path and platform tools.
3. Accept the licences with flutter doctor --android-licenses.
4. Run flutter doctor and confirm the Android toolchain checks.
5. Use a connected device or emulator when running the application.

# 4. Flutter Development Environment Setup

## Visual Studio Code

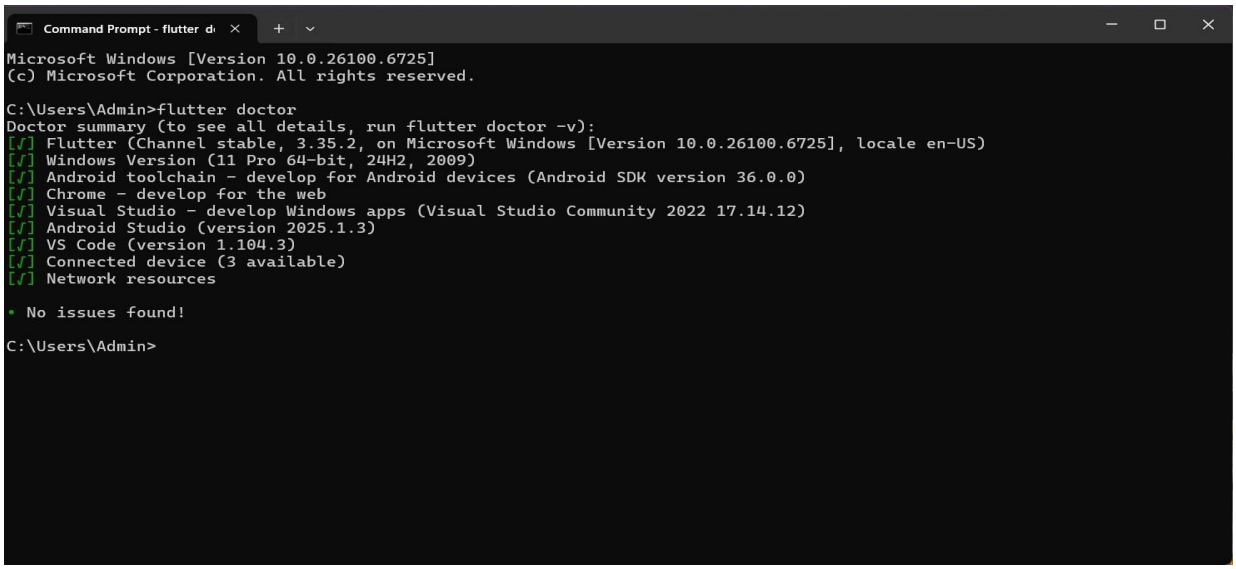
1. Install Visual Studio Code.
2. Open the Extensions panel with Ctrl + Shift + X.
3. Install the Flutter extension; Dart is installed as a depen
dency.
4. Open the project folder in VS Code.
5. Use the integrated terminal for Flutter commands and d
ebugging.




Dart and Flutter extensions installed in Visual Studio Code

## Verify the Environment

```
flutter --version
dart --version
flutter doctor
flutter devices
```




Flutter doctor output confirming the available toolchain

## Environment Checks

- Confirm that the Flutter SDK and Dart versions are available.
- Resolve any missing Android SDK or platform-tool warnings.
- Check that a device or emulator is listed before running the app.
- Use a clean terminal after changing PATH environment variables.

## Useful Commands

```
flutter doctor -v
flutter clean
flutter pub get
flutter analyze
flutter run
```

The verification screenshots on this and the preceding pages document the developme nt environment.
They do not represent screens inside the Electricity Usage Tracker application.

# 5. Dart Programming Fundamentals Used in the Project

Dart provides the programming foundation for the Flutter application. The project uses variables, classes, lists, maps, functions, conditional statements, loops, asynchronous programming, and null safety.

## Variables and Data Types

```
String deviceName = "Home Meter";
int userId = 101;
double tariff = 7.0;
bool isDark = true;
List<double> readings = [120.5, 145.0];
Map<String, dynamic> user = {"name": "Dileep"};
```

## Conditional Statements

```
if (current < previous) {
  showMessage("Invalid reading");
} else {
  units = current - previous;
}
```

## Lists

Lists hold collections such as readings, chart entries, and dashboard cards.

```
final readings = <double>[120.5, 145.0];

for (final units in readi
ngs) {
  print("Units: $units");
}
```

## Classes and Models

```
class ElectricityReading {
  final int? id;
  final int userId;
  final double previousReading;
  final double currentReading;
  final double units;
  final double billAmount;
  final String date;

  ElectricityReading({
    this.id,
    required this.userId,
    required this.previousReading,
    required this.currentReading,
    required this.units,
    required this.billAmount,
    required this.date,
  });

  Map<String, dynamic> toMap() => {
        "id": id,
        "user_id": userId,
        "previous_reading": previousReading,
        "current_reading": currentReading,
        "units": units,
        "bill_amount": billAmount,
        "date": date,
      };
}
```

## Asynchronous Programming

Database operations are asynchronous so that the interface remains responsive while SQLite reads or writes data.

```
Future<List<ElectricityReading>> loadReadings(int use
rId) async {
  final db = await DatabaseService.instance.database;
  final rows = await db.query(
    "readings",
    where: "user_id = ?",
    whereArgs: [userId],
    orderBy: "id DESC",
  );
  return rows.map(ElectricityReading.fromMap).toList(
);
}
```

# 6. Creating and Configuring the Flutter Project

## Creating the Application

```
flutter create electricity_usage
_tracker
cd electricity_usage_tracker
code .
flutter run
```

## pubspec.yaml

The pubspec declares the Flutter SDK and the packages used by the application. The current project depends on sqflite, path, fl_chart, shared_preferences, and cupertino_icons.

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  sqflite: ^2.4.3
  path: ^1.9.1
  fl_chart: ^1.2.0
  shared_preferences: ^2.5.6
```

## Main Application Entry Point

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ElectricityUsageTracker());
}

class ElectricityUsageTracker extends StatefulWidget {
  const ElectricityUsageTracker({super.key});

  @override
  State<ElectricityUsageTracker> createS
tate() =>
      _ElectricityUsageTrackerState();
}
```

# 7. Electricity Usage Tracker – Case Study

The Electricity Usage Tracker is the central case study for this Flutter documentation. It is designed for a user who wants to record meter readings, monitor consumption, understand monthly trends, and estimate electricity costs without relying on a cloud account.

## Core Features

- User registration, login, and local account storage.
- Profile viewing, personal-information editing, and tariff configuration.
- Meter-reading entry with previous and current values.
- Automatic calculation of consumed units and estimated bill.
- Usage history with single-item deletion and clear-all support.
- Monthly usage and bill charts with averages and an insight message.
- In-app high-usage alerts, reminders, and energy-saving guidance.
- Light and dark themes with a persisted user preference.
- Responsive layouts for different mobile screen sizes.
- Local SQLite persistence for users, readings, and settings.

## User Journey

1. The user opens the splash screen and continues from the la
nding page.
2. A new user registers with a name, email, and password.
3. The user signs in and reaches the dashboard.
4. The dashboard displays the latest usage, bill, and previous
usage.
5. The user adds a meter reading with previous and current va
lues.
6. The app calculates units and the estimated bill immediately.
7. The reading is stored locally and appears in usage history.
8. The user reviews monthly charts and the energy insight.
9. The user checks alerts or updates the tariff in the profile.
10. Logout clears the navigation stack and returns to login.

# 7. Electricity Usage Tracker – Case Study

## Technologies Used

| Technology | Role in the Project |
|---|---|
| Flutter | Cross-platform UI, navigation, and widget composition. |
| Dart | Application language and model classes. |
| sqflite | Asynchronous SQLite access on mobile devices. |
| SQLite | Local storage for users, readings, and tariff settings. |
| fl_chart | Bar and line charts in the analytics screen. |
| shared_preferences | Persisted light or dark theme selection. |
| Material 3 | Cards, forms, dialogs, and responsive layout. |
| Visual Studio Code | Development and debugging environment. |

## System Overview

```
Presentation
  MaterialApp, Splash, Landing, Login/Register
  Dashboard and feature screens
        ↓
UI state
  setState, AnimationController, ThemeController
        ↓
Data access
  DatabaseHelper (users) / DatabaseService (readings)
        ↓
Storage
  electricity_tracker.db + electricity_usage.db
```

Technology and data-flow overview of the application

# 8. User Authentication and Profile Management

Registration creates a local account, while login verifies the entered email and password against the users table. Authentication is handled by DatabaseHelper rather than a remote service.

## Registration

- All fields are required.
- The email must match a simple email pattern.
- The password must contain at least six characters.
- The password and confirmation must match.
- Duplicate email addresses are rejected before insertion.

## Login

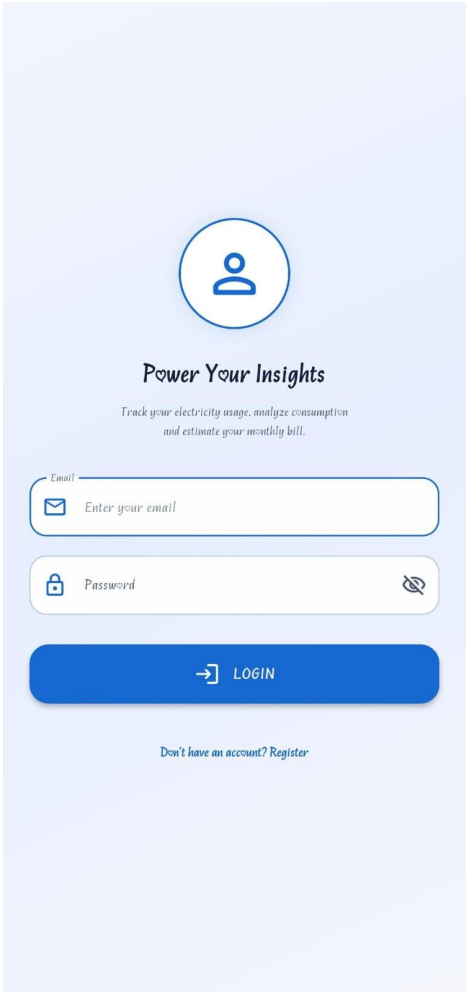
The login screen validates empty input, queries the local users table, and opens the dashboard only when matching credentials are found.

```
final user = await DatabaseHelper.instance.loginUser(
  email: email,
  password: password,
);

if (!mounted) return;

if (user == null) {
  showMessage("Invalid email or password.");
  return;
}

Navigator.pushReplacement(
  context,
  MaterialPageRoute(
    builder: (_) => DashboardScre
en(
      userId: user["id"] as int,
      userName: user["name"] as S
tring,
      userEmail: user["email"] as
 String,
    ),
  ),
);
```




**LOGIN SCREEN**

## Profile Management

The profile screen displays the signed-in user's name and email, provides access to personal-information editing, and manages the electricity tariff and appearance settings.

- View the current account name and email address.
- Edit the name and email through a separate screen.
- Prevent an update to an email address that already exists.
- Configure the tariff used for new reading calculations.
- Choose between the dark and light themes.

Updated profile values are returned to the dashboard so that the welcome section reflect s the new name
without requiring the user to sign in again.

# 9. Meter Reading Management

A meter reading records the previous meter value, the current meter value, the calculated units, the tariff used, the estimated bill, the owning user, and the date. Readings are the main data source for the dashboard, history, analytics, and alerts.

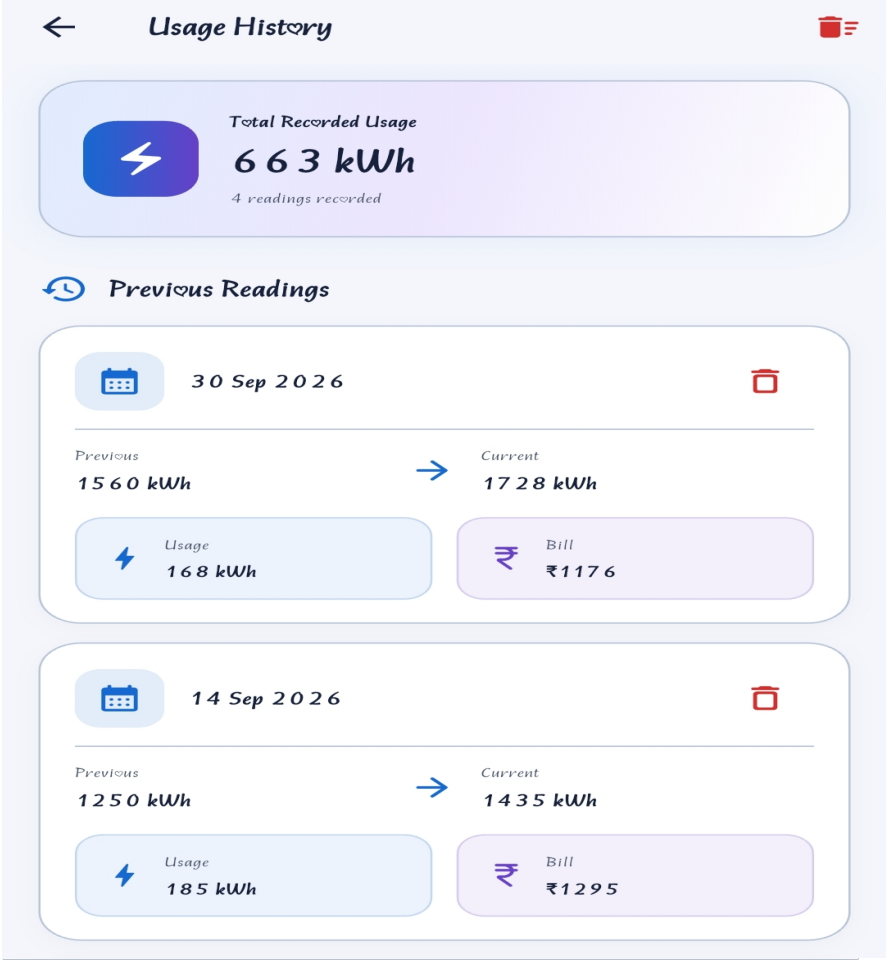
## Reading Management Operations

- Add a reading for the signed-in user.
- Reject non-numeric, negative, or decreasing meter values.
- Calculate units and the estimated bill before saving.
- View readings in reverse chronological order.
- Delete one reading or clear all readings after confirmation.

## Reading Card

```
Card(
  child: ListTile(
    leading: const Icon(Icons.bolt_rounded),
    title: Text(
      "${item.units.toStringAsFixed(1)} kWh",
    ),
    subtitle: Text(
      "${item.previousReading} → "
      "${item.currentReading} kWh",
    ),
    trailing: IconButton(
icon: const Icon(Icons.delete_outlin
e),
onPressed: onDelete,
),
),
)
```

The history screen also presents the previous and current readings and the calculated bill beneath each card. Delete operations use a confirmation dialog to avoid accidental data loss.




## Reading Ownership

Each saved reading contains a user_id value. DatabaseService filters every query by the current user, so the history, analytics, and alert screens receive only that user's readings.

- One user can own many readings.
- Each reading belongs to exactly one user.
- Readings store their own date and calculated values.
- Deletion affects only the selected reading unless clear-all is confirmed.

This is a one-to-many relationship. Unlike a many-to-many skill model, the applic ation does not
require a separate reading-membership table.

# 10. Consumption and Bill Estimation

Consumption is calculated from the difference between the current and previous meter values. The estimated bill is the consumed units multiplied by the tariff. Both values are stored with the reading so that historical bills remain consistent if the tariff changes later.

## Calculation Operations

- Read the previous value, current value, and tariff.
- Reject empty, non-numeric, or negative input.
- Reject a current reading below the previous reading.
- Calculate units as current minus previous.
- Calculate the estimated bill as units multiplied by tariff.

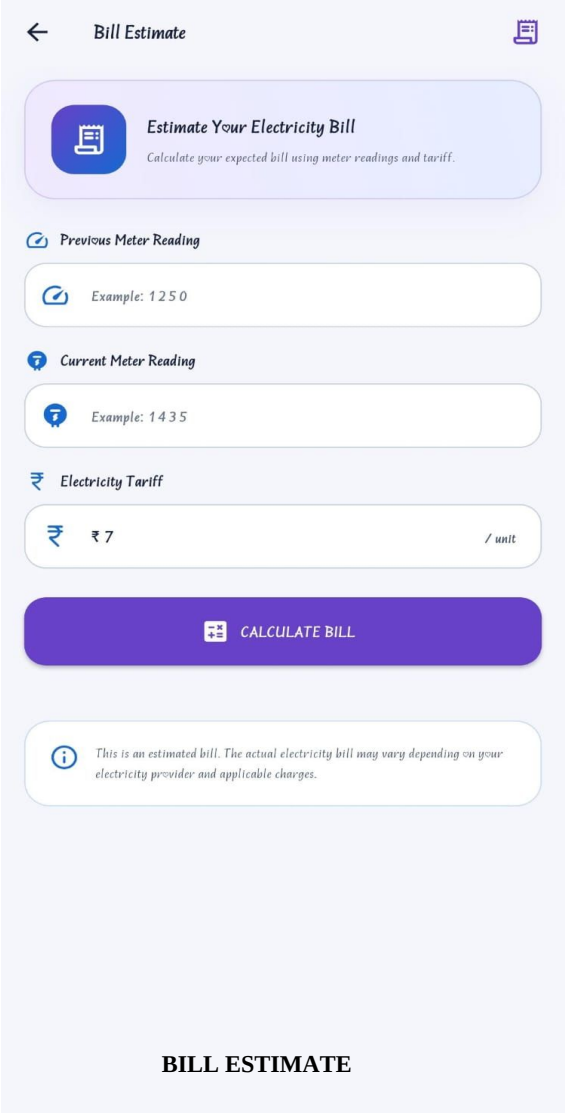
## Calculation Code

```
void calculateUsage() {
  final values = readInputs();
  if (values == null) return;

  setState(() {
units = values.current - values.prev
ious;
estimatedBill = units * values.rate;
});
}
```

## Worked Example

| Value | Example | Result |
|---|---|---|
| Previous reading | 1200 | Starting value |
| Current reading | 1285 | 85 units consumed |
| Tariff | ₹8.00 | ₹680.00 estimated bill |




**BILL ESTIMATE**

## Meter Reading vs Bill Estimate

| Feature | Behaviour |
|---|---|
| Purpose | A reading records actual consumption; the estimate is a quick what-if calculation. |
| Storage | Readings are saved; estimates are temporary and are not inserted. |
| Input | Reading uses previous, current, and tariff; estimate accepts the same values independently. |
| Result | Both display units and a bill, but only a saved reading affects history and analytics. |

# 11. Usage Analytics and History

The analytics screen reads the user's saved readings, sorts them by date, and presents summary metrics, a bar chart for monthly consumption, a line chart for bill amounts, and a short insight based on the latest two readings.

## Summary Metrics

- Average units across all saved readings.
- Average estimated bill across all saved readings.
- Highest-usage reading and its date.
- Latest-versus-previous usage insight.

## Analytics Flow

1. Load the current user's readings from DatabaseService.
2. Sort the records by date in ascending order.
3. Calculate average usage and average bill.
4. Identify the highest-usage reading.
5. Build the usage bar chart and bill line chart.
6. Compare the latest two readings for the insight.

## Average Calculation

```
double get averageUsage {
  if (readings.isEmpty) return 0;

  return readings.fold<double>(
        0,
        (total, reading) => total + readi
ng.units,
      ) /
      readings.length;
}
```

History is separate from analytics: the history screen focuses on individual records and deletion, while analytics aggregates the same data into trends and comparisons.

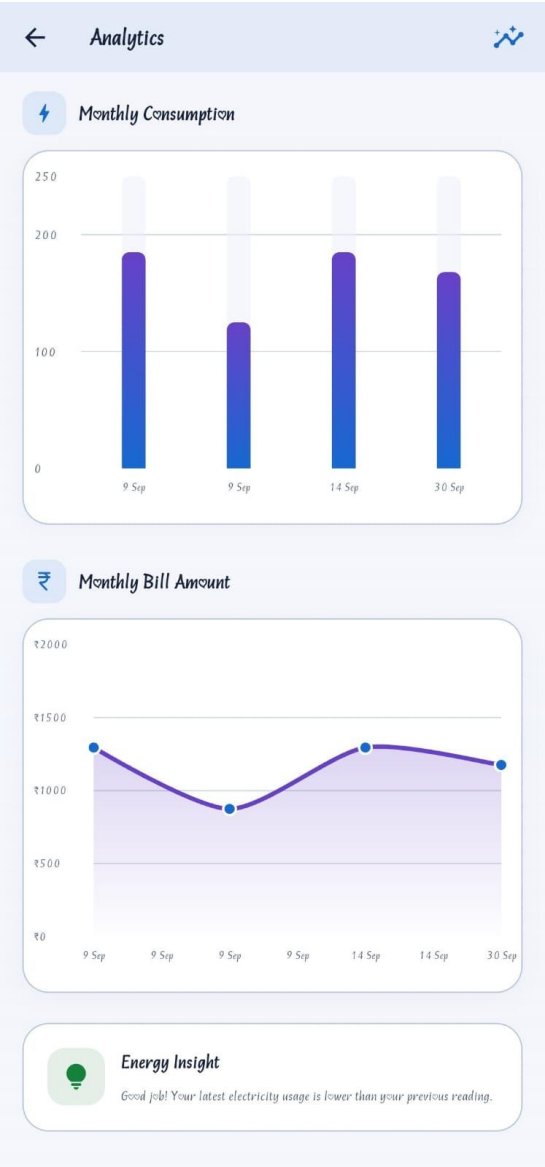
## History and Chart Interpretation

The usage history screen lists every saved reading with its date, meter values, consumed units, and bill. The analytics screen uses the same records to compare consumption over time.

## Chart Interpretation

- A rising bar indicates higher consumption in a later reading.
- A falling line indicates a lower estimated bill.
- The average cards provide a quick baseline for comparison.
- The highest-usage card identifies the peak record and date.

Charts are generated locally with fl_chart. No chart data is uploaded to an external an alytics service.




Analytics screen with summary cards, charts, and insight

# 12. Alerts and Energy-Saving Guidance

Alerts are presented inside the application. The screen loads the latest readings and compares the newest consumed units with the previous reading. It does not schedule operating-system notifications or send messages to another device.

## High-Usage Check

```
bool get hasHighUsage {
  final latest = latestReading;
  final previous = previousReading;

  if (latest == null || previous == null) return false;

  return latest.units > previous.units;
}
```

## Alert Types

- High-usage warning when the latest units exceed the previous units.
- Monthly reminder to add a new meter reading.
- Bill update showing the latest estimated amount.
- Energy-saving tip for reducing high-power appliance use.

If fewer than two readings exist, the comparison is unavailable and the screen shows a helpful empty
or caught-up state instead of a false warning.

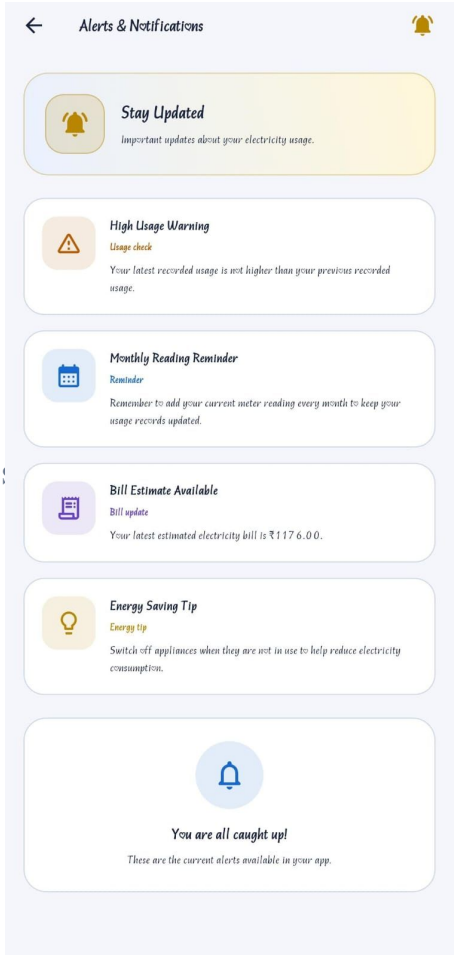
## Alert Presentation

The alerts screen uses theme-aware cards and a compact empty state. A refresh action reloads readings after new data has been added.

## Alert States

| State | Meaning |
|---|---|
| High usage | The latest reading has more units than the previous reading. |
| Reminder | The user is encouraged to keep meter records current. |
| Bill update | The latest estimated bill is displayed. |
| Caught up | No warning is required, or there are too few readings to compare. |

These messages are informational guidance, not electricity-provider notices. The esti mated bill may
differ from the actual bill because taxes, fixed charges, and slab rates are not modelle d.




Alerts and energy-saving guidance

# 13. Exploring Flutter Widgets in the Project

Flutter widgets are the building blocks of every screen. The project combines standard Material widgets with private builder methods and custom painters to create a consistent energy-themed interface.

## Important Widgets

| Widget | Use in the Project |
|---|---|
| MaterialApp | Root configuration, themes, and initial route. |
| Scaffold | Base structure for each screen. |
| AppBar | Titles, back actions, alerts, and clear-history action. |
| Text | Names, readings, units, bills, and messages. |
| TextField | Email, password, meter values, and tariff input. |
| Card / Container | Reading cards, stat cards, and action tiles. |
| ListView | Dashboard sections, history, and alerts. |
| Column / Row | Vertical sections and paired summary cards. |
| Expanded | Balanced stat cards without horizontal overflow. |
| SingleChildScrollView | Scrollable forms and profile content. |
| CustomPaint | Animated energy backgrounds on major screens. |
| AnimatedBuilder | Rebuilds animated backgrounds efficiently. |
| CircularProgressIndicator | Loading and splash feedback. |

The screens avoid fixed screen coordinates where possible and use padding, spacin
g, and flexible
containers to remain readable at different widths.

## Example Widget Tree

```
ElectricityUsageTracker
 └── MaterialApp
      └── SplashScreen
           └── LandingScreen
                └── LoginScreen
                     └── DashboardScreen
                          ├── AddReadingScreen
                          ├── UsageHistoryScreen
                          ├── AnalyticsScreen
                          ├── BillEstimateScreen
                          ├── AlertsScreen
                          └── ProfileScreen
```

```
DashboardScreen
 └── Scaffold
      ├── AppBar (alerts, profile)
      └── Body
           └── Stack
                ├── CustomPaint background
                └── RefreshIndicator
                     └── ListView
                          ├── Welcome header
                          ├── Main usage card
                          ├── Stat cards
                          ├── Quick actions
                          ├── Comparison card
                          └── Latest readings
```

Dashboard widget composition in the Electricity Usage Tracker

The tree mirrors the source structure: the dashboard owns its animated background, ref resh behaviour,
and scrollable content, while feature screens are opened as separate routes.

# 14. Responsive UI Design

Responsive design keeps content usable across different Android screen sizes. The application relies on flexible layout widgets, scrollable content, and theme-aware spacing rather than fixed screen coordinates.

## Responsive Techniques

- SafeArea keeps content away from system bars and cutouts.
- Expanded and Flexible share horizontal space without overflow.
- ListView handles long dashboard, history, and alert content.
- SingleChildScrollView protects forms on shorter screens.
- MediaQuery.sizeOf is used for height-sensitive alert content.
- Consistent padding and spacing maintain readability.

## Dashboard Layout Pattern

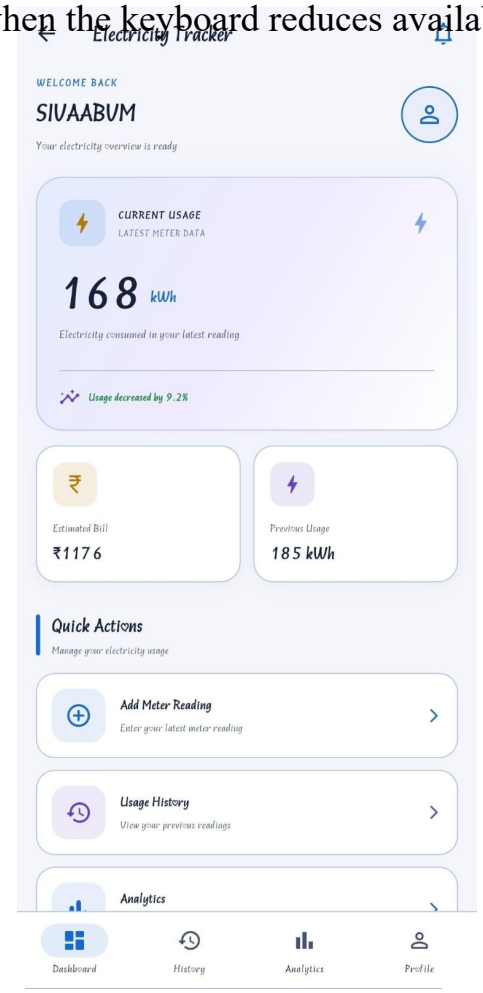
The dashboard uses a refreshable ListView and pairs summary cards inside a Row with Expanded children. This lets both cards share the available width safely.

```
RefreshIndicator(
  onRefresh: loadDashboard,
  child: ListView(
    padding: const EdgeInsets.fromLTRB(18, 12
, 18, 28),
    children: [
      _buildWelcomeSection(),
      _buildMainUsageCard(),
      Row(
        children: [
          Expanded(child: _statCard(...)),
          const SizedBox(width: 12),
          Expanded(child: _statCard(...)),
        ],
      ),
    ],
  ),
)
```

## Project Application

The dashboard, history list, analytics charts, alerts, and forms must remain accessible when device width or height changes. The interface therefore avoids depending on fixed positions for core content.

- Long content is scrollable rather than clipped.
- Card rows use Expanded children for balanced widths.
- MediaQuery is used only where a proportional height is useful.
- Forms remain usable when the keyboard reduces availabl e space.




**Responsive dashboard layout**

The placeholder above should be replaced with a screenshot of the Electricity U sage Tracker
dashboard. The surrounding description already reflects the implemented respon sive structure.

# 15. Navigation Between Screens

Navigation connects the authentication flow, dashboard, and feature modules. The application uses MaterialPageRoute for feature screens and replaces the login route after successful authentication.

## Main Navigation Flow

```
SplashScreen
  ↓
LandingScreen
  ↓
LoginScreen ↔ RegisterScreen
  ↓
DashboardScreen
  ├── AddReadingScreen
  ├── UsageHistoryScreen
  ├── AnalyticsScreen
  ├── BillEstimateScreen
  ├── AlertsScreen
  └── ProfileScreen
        └── PersonalInformatio
nScreen
```

## Logout

Logout clears the authenticated navigation stack so that the user cannot return to the dashboard with the back button.

```
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(
    builder: (_) => const LoginS
creen(),
  ),
  (route) => false,
);
```

The dashboard also reloads its readings and profile values after returning from feature screens, keeping the visible summary current.

**Logout → Login**
**Navigation stack cleared**

# 16. Stateful and Stateless Widgets

A StatelessWidget is used when a widget does not maintain mutable state. A StatefulWidget is used when the interface must change after a database read, form submission, theme update, or animation tick.

## Stateless Widgets

The landing screen and small reusable presentation pieces are stateless where no mutable data is required. They receive values and callbacks through their constructors.

## Stateful Widgets

- Login and registration screens maintain form and loading state.
- The dashboard maintains readings, the current name, and loading state.
- Add Reading and Bill Estimate maintain entered values and results.
- Analytics, history, alerts, and profile load and refresh local data.

## setState Example

```
Future<void> loadDashboard() async {
  try {
    final data = await DatabaseService.instance.g
etReadings(
      widget.userId,
    );
    if (!mounted) return;
    setState(() {
      readings = data;
      isLoading = false;
    });
  } catch (_) {
    if (!mounted) return;
    setState(() => isLoading = false);
  }
}
```

The mounted checks prevent state updates after a scree
n has been removed from the widget tree.

# 17. State Management

The project uses local screen state for readings and forms, and a singleton ChangeNotifier for application-wide theme state. It does not use Provider or a remote authentication session.

## Why State Management Is Required

- The selected theme affects every screen in the MaterialApp.
- The dashboard must refresh after readings are added or deleted.
- Profile changes must flow back to the dashboard name.
- Loading and error states must be reflected in the interface.

## Current Approach

ThemeController extends ChangeNotifier and is observed by AnimatedBuilder in main.dart. Screens observe it directly when they expose theme choices. User and reading data are reloaded from SQLite rather than stored in a global provider.

```
class ThemeController extends ChangeNotifier {
  ThemeController._();
  static final ThemeController instance = ThemeController._();

  ThemeMode _themeMode = ThemeMode.dark;
  ThemeMode get themeMode => _themeMode;

  Future<void> loadTheme() async {
    final preferences = await SharedPreferences.getInstance();
    final savedTheme = preferences.getString("app_theme_mode");
    _themeMode = savedTheme == "light"
        ? ThemeMode.light
        : ThemeMode.dark;
    notifyListeners();
  }

  Future<void> setTheme(ThemeMode mode) async {
    final preferences = await SharedPreferences.ge
tInstance();
    await preferences.setString(
      "app_theme_mode",
      mode == ThemeMode.light ? "light" : "dark",
    );
    _themeMode = mode;
    notifyListeners();
  }
}
```

This approach is sufficient for a mostly local applicati
on, but a larger version would benefit from a
shared session or repository layer.

# 18. Custom Widgets and Styling

Reusable UI patterns reduce duplication and keep the visual language consistent. The project uses private builder methods for repeated cards and sections, custom painters for energy backgrounds, and shared ThemeData for colours and input styles.

## Reusable UI Patterns

- Stat cards for estimated bill and previous usage.
- Action cards for adding readings, viewing history, and analytics.
- Section headers for dashboard and profile groups.
- Reading cards for history entries.
- Profile option tiles for account and appearance settings.
- Custom painters for animated backgrounds.

## Styling Principles

- Use the dark cyber and premium light palettes consistently.
- Use rounded cards, dialogs, and input borders.
- Keep primary actions visually distinct from destructive actions.
- Maintain readable contrast in both themes.

## Theme Configuration

```
static final ThemeData darkTheme = Them
eData(
  useMaterial3: true,
  brightness: Brightness.dark,
  scaffoldBackgroundColor: background,
  colorScheme: const ColorScheme.dark(
    primary: electricBlue,
    secondary: cyberPurple,
    surface: surface,
  ),
  elevatedButtonTheme: ElevatedButtonTh
emeData(
    style: ElevatedButton.styleFrom(
      backgroundColor: electricBlue,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circ
ular(16),
      ),
    ),
  ),
);
```

**Dark Cyber Theme**

#070B14 #101827 #00B7FF #8B5CF6 #FFD60A

**Premium Light Theme**

#F4F6FC #FFFFFF #1769D2 #6741C8 #FFC928

**APPLICATION ICON**



Electricity
Usage Tracker

# 19. Forms and Validation

Forms are used for registration, login, meter readings, bill estimation, profile editing, and tariff changes. Validation prevents invalid values from reaching SQLite and gives the user a clear message when input cannot be accepted.

## Validation Requirements

| Form | Validation |
|---|---|
| Registration | Required fields, email pattern, minimum password length, matching confirmation. |
| Login | Both email and password must be entered; credentials must match. |
| Personal Information | Name and email are required; email must contain @ and remain unique. |
| Add Reading | Numeric, non-negative values; current cannot be below previous. |
| Bill Estimate | Numeric, non-negative values and a positive tariff. |
| Tariff Dialog | A finite amount greater than zero. |

## Registration Validation

```
final name = nameController.text.trim();
final email = emailController.text.trim();
final password = passwordController.text;

if (name.isEmpty || email.isEmpty || password.isEmpty) {
  _showMessage("Please fill all fields");
  return;
}

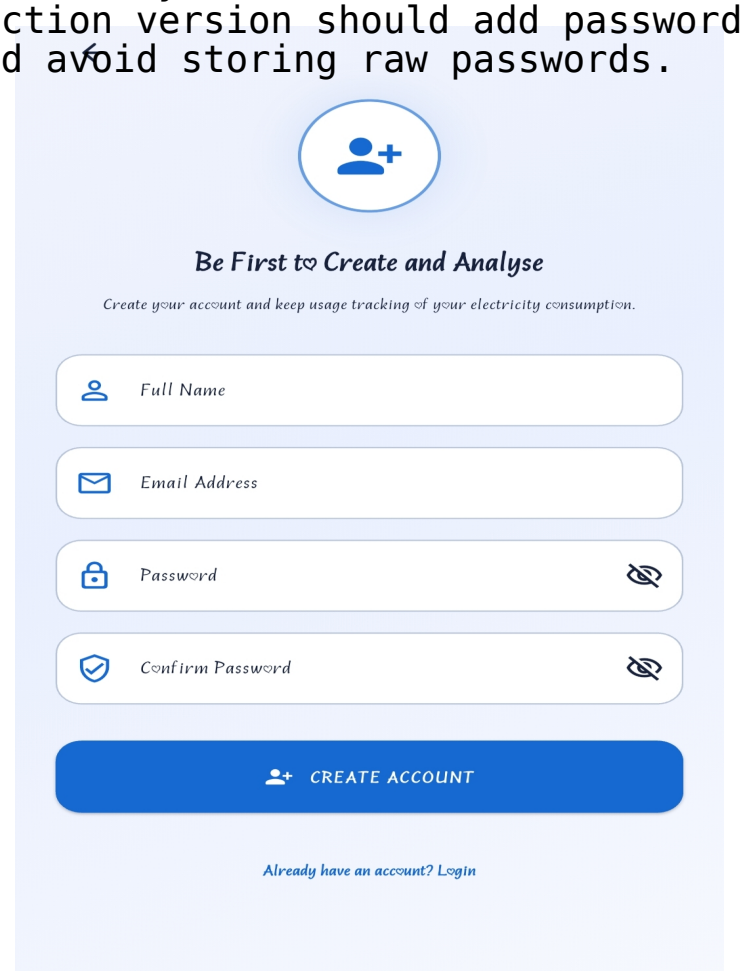
if (!RegExp(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
    .hasMatch(email)) {
  _showMessage("Please enter a valid email
address");
  return;
}
```

## Submission

```
if (name.isEmpty || email.isEmpty) {
  _showMessage("Please fill in all fields.");
  return;
}

await DatabaseHelper.instance.insertUser(
name: name,
email: email,
password: passwor
d,
);
```

Client-side validation improves usability, but it is not a security boundary. This project performs validation locally and stores credentials locally as well. A production version should add password hashing and avoid storing raw passwords.




Registration form with validation feedback

The form placeholder should be replaced with a screenshot showing the actual registration fields and validation message. The current project does not use a remote backend for submission.

# 20. Local Data Storage with SQLite

The application persists data on the device with sqflite. There is no REST API, Java backend, or cloud database. Account information and readings are stored in two SQLite database files.

## Local Persistence

- DatabaseHelper manages the electricity_tracker.db user and reading tables.
- DatabaseService manages electricity_usage.db readings and settings.
- Database access is asynchronous and uses parameterised queries.
- Records are filtered by user_id where appropriate.
- The tariff is stored in a settings row with id 1.

## Database Files

| File | Contents |
|---|---|
| electricity_tracker.db | users and a legacy readings table used by DatabaseHelper. |
| electricity_usage.db | readings and settings used by the dashboard and feature screens. |

## Database Initialisation

```
Future<Database> _initDatabase() async
{
  final databasePath = await getDatabas
esPath();
  final path = join(
    databasePath,
    "electricity_usage.db",
  );
  return openDatabase(
    path,
    version: 3,
    onCreate: _createDatabase,
    onUpgrade: _upgradeDatabase,
  );
}
```

## Data Access Layering

```
Widget or screen action
        ↓
setState / ThemeController
        ↓
DatabaseHelper or DatabaseService
        ↓
sqflite query, insert, update, or delete
        ↓
SQLite file on the device
        ↓
Dart model
        ↓
Updated interface
```

## Error Handling

- Show a loading indicator while asynchronous work is running.
- Catch database failures and avoid leaving the screen stuck in loading state.
- Display a short, meaningful message instead of a raw exception.
- Refresh the visible data only after a successful operation.
- Use confirmation dialogs before destructive actions.

DatabaseHelper and DatabaseService are both singletons that cache the opened database. Screens await their operations and update local state after the future completes.

## Data Integrity Notes

- Readings are inserted with calculated units and bill values.
- Deletion is scoped to a selected id unless clear-all is confirmed.
- The tariff update replaces the single settings row.
-

The two database files create a maintenance risk that should be consolidated in a l ater version.

# 21. Database and Data Models

The project uses SQLite tables for users, readings, and settings. The schema is intentionally small, but two separate data layers currently define overlapping reading information.

## Database Tables

| Table / File | Important Columns |
|---|---|
| users / tracker | id, name, email (unique), password |
| readings / tracker | user_id, previous_reading, current_reading, units, tariff, estimated_bill, date |
| readings / usage | user_id, previous_reading, current_reading, units, bill_amount, date |
| settings / usage | id, tariff |

## Conceptual Relationships

```
users 1 ──────< readings  (DatabaseHelper)

users 1 ──────< readings  (DatabaseService)

settings 1 ─── tariff    (global default: 7.0)
```

There is no many-to-many relationship or connection-request model in this application. A reading belongs to one user, and the settings table stores a single global tariff.

## Model Responsibilities

- ElectricityReading carries reading values between the UI and database.
- toMap converts the model into a SQLite-compatible map.
- fromMap converts a database row back into a typed model.
- The model does not perform calculation; screens and services provide values.

## Dart Model

```
class ElectricityReading {
  final int? id;
  final int userId;
  final double previousReading;
  final double currentReading;
  final double units;
  final double billAmount;
  final String date;

  ElectricityReading({
    this.id,
    required this.userId,
    required this.previousReading,
    required this.currentReading,
    required this.units,
    required this.billAmount,
    required this.date,
  });

  Map<String, dynamic> toMap() => {
        "id": id,
        "user_id": userId,
        "previous_reading": previousReading,
        "current_reading": currentReading,
        "units": units,
        "bill_amount": billAmount,
        "date": date,
      };
}
```

The fromMap factory casts numeric SQLite values through num and calls toDouble, which handles integer values returned by SQLite for whole-number REAL columns.

## Schema Improvement

- Consolidate the duplicate reading tables into one source of truth.
- Associate the tariff with a user if per-user rates are required.
- Add indexes on user_id and date for larger histories.
- Store password hashes instead of plain-text passwords.

## 22. Animations and User Experience

Animations are used to reinforce the energy theme and provide feedback. They are deliberately lightweight: no quiz-result effects, leaderboards, or complex page transitions are required.

### Suitable Animations

- Slow custom-painted energy backgrounds on major screens.
- Circular progress indicators during loading and authentication.
- AnimatedContainer feedback for theme selection.
- Button and card ripple feedback for user actions.
- Smooth refresh behaviour after readings are added or deleted.

### Animation Controller

```
late final AnimationController _animationController;

@override
void initState() {
  super.initState();
  _animationController = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 11),
  )..repeat();
}
```

AnimatedBuilder rebuilds only the custom-painted background while the contr
oller runs. The
controller is disposed when the screen is removed to avoid unnecessary work.

# 23. Testing and Debugging

Testing should verify calculations, validation, persistence, navigation, and the visual states of each screen. The current repository does not yet contain a complete test suite for the implemented application.

## Current Test Status

| Area | Current Status |
|---|---|
| Widget test | Only the default counter test remains; it does not test this app. |
| Unit tests | No dedicated tests for calculation or validation. |
| Integration tests | No automated integration suite. |
| Manual testing | Requires an Android device or emulator for sqflite. |

## Recommended Tests

- Unit-test unit and bill calculations, including invalid readings.
- Test registration email validation and duplicate-email rejection.
- Test ElectricityReading toMap and fromMap conversions.
- Widget-test login fields, reading forms, and validation messages.
- Integration-test add reading → history → analytics → alerts.
- Replace the default widget_test.dart with application-specific tests.

## Debugging Tools

- Flutter DevTools for widget inspection and performance checks.
- The Visual Studio Code debugger for breakpoints and stepping.
- debugPrint and console output for development diagnostics.
- flutter analyze for static analysis and lint feedback.
- Device or emulator logs for runtime and SQLite failures.

## Error Handling Example

```
Future<void> loadAnalytics() async {
  try {
    final data = await DatabaseService.instance.getReadings(
      widget.userId,
    );
    data.sort((a, b) => a.date.compareTo(b.date));
    if (!mounted) return;
    setState(() {
      readings = data;
      isLoading = false;
    });
  } catch (_) {
    if (!mounted) return;
    setState(() => isLoading = false);
    _showMessage("Unable to load analytics.");
  }
}
```

The catch block stops the loading state and shows a short message. It does not expo
se the internal
exception to the user.

# 24. Application Workflow

The complete workflow moves from splash and landing screens through authentication to the dashboard, where the user can add readings, view history, analyse trends, estimate bills, check alerts, and manage the profile.

## Complete Screen Workflow

```
SplashScreen
  ↓
LandingScreen
  ↓
LoginScreen ⇔ RegisterScreen
  ↓
DashboardScreen
  ├── AddReadingScreen
  ├── UsageHistoryScreen
  ├── AnalyticsScreen
  ├── BillEstimateScreen
  ├── AlertsScreen
  └── ProfileScreen
       ├── PersonalInformationScreen
       ├── Tariff dialog
       └── Logout → LoginScreen
```

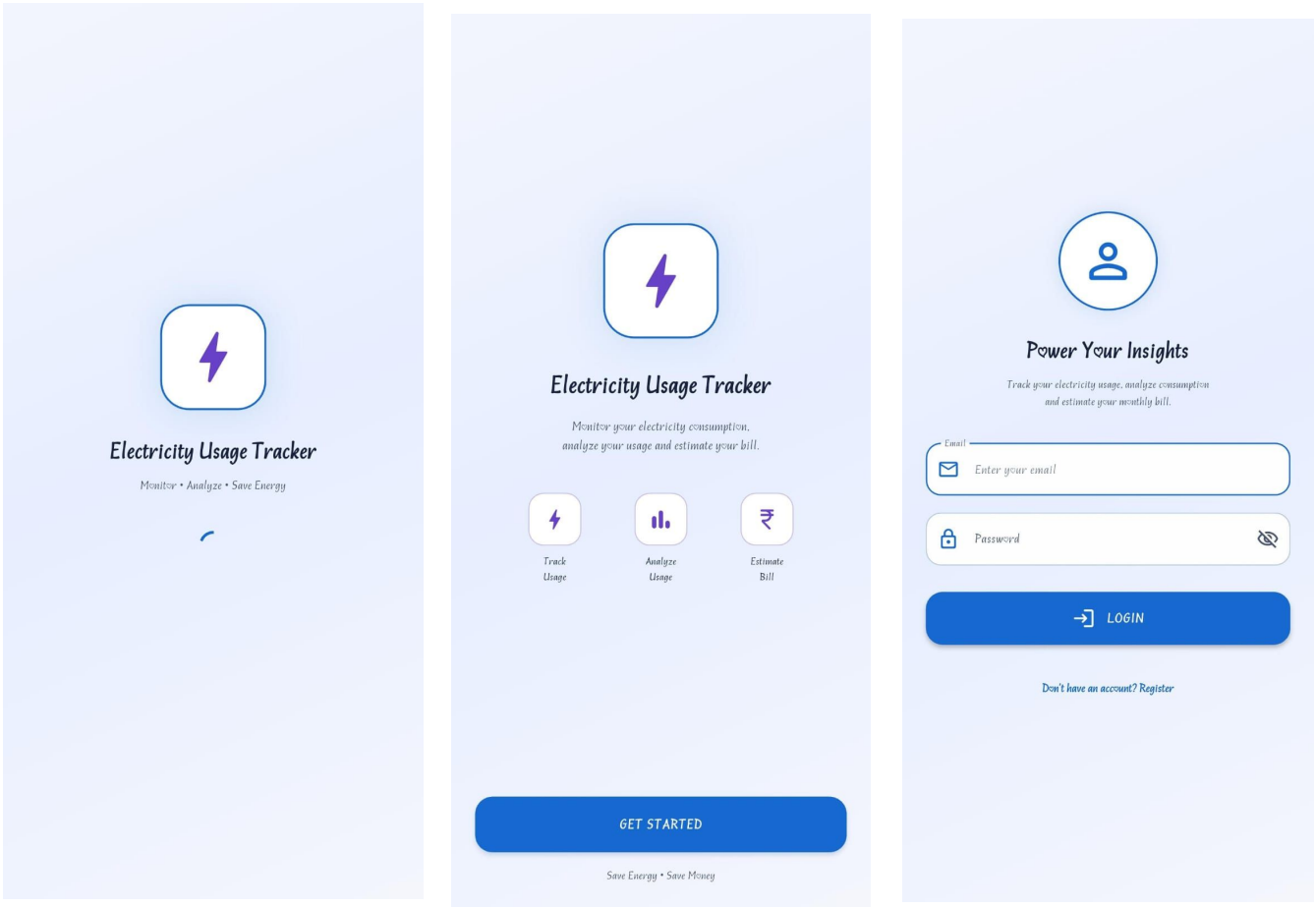
## Dashboard Responsibilities

- Provide the central starting point after authentication.
- Show the latest units, estimated bill, and previous usage.
- Open reading, history, analytics, bill, alert, and profile screens.
- Reload data after returning from a feature screen.
- Keep loading and empty states clear without duplicating feature logic.

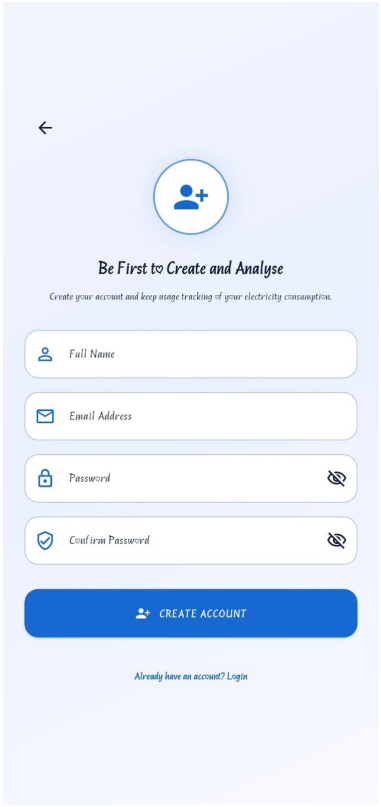
## Data Flow

```
User Action
    ↓
Flutter Widget
    ↓
setState / ThemeController
    ↓
DatabaseHelper / DatabaseService
    ↓
SQLite query or mutation
    ↓
Dart Model
    ↓
Updated UI
```

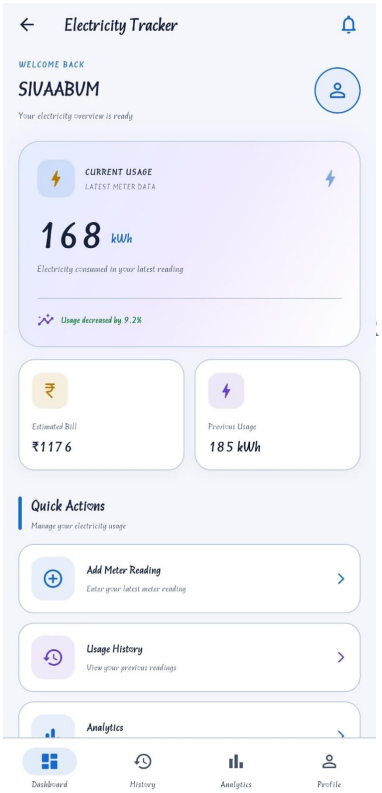
# Application Screens



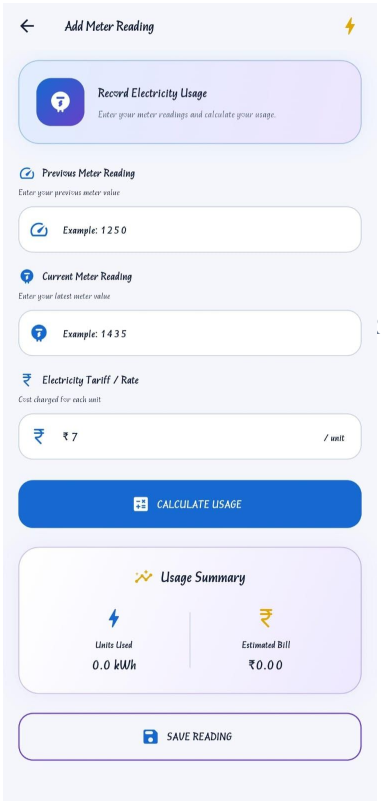

Splash, landing, and login screens



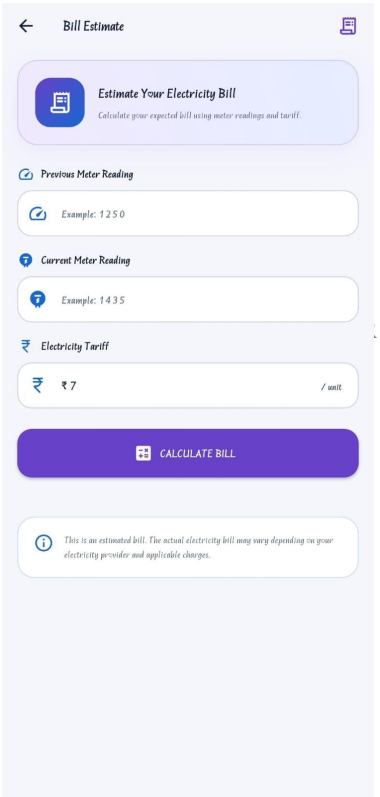

**REGISTER_SCREEN**



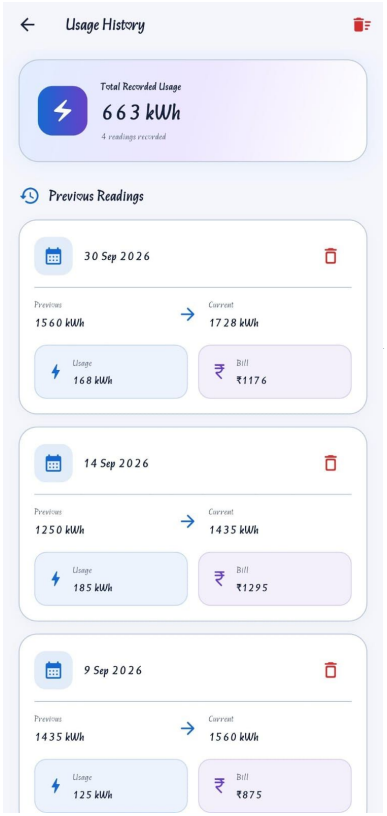

**DASHBOARD_SCREEN**



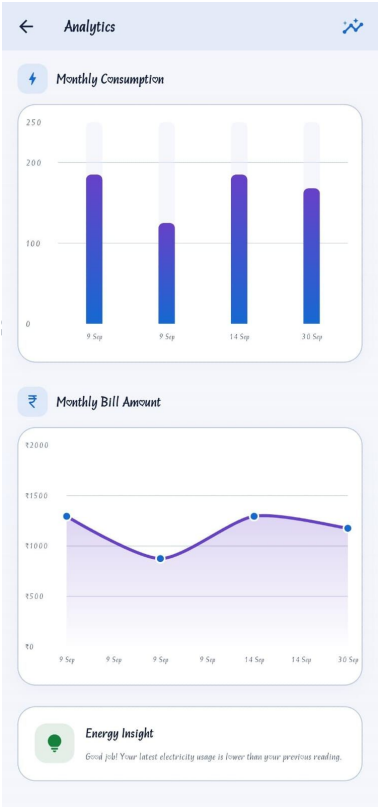

**ADD METER READING_SCREEN**



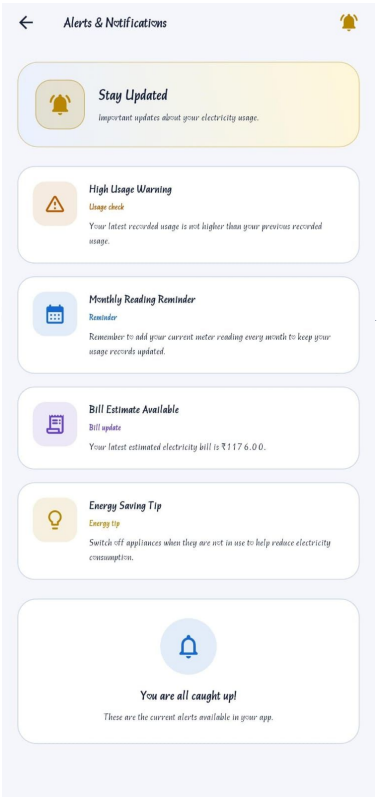

**BILL ESTIMATE_SCREEN**



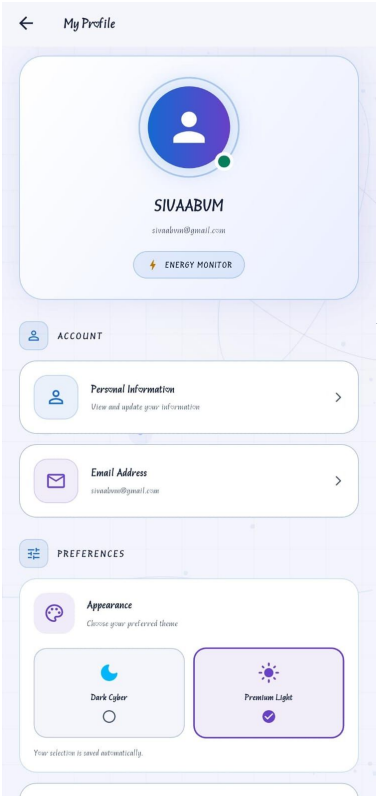

**HISTORY_SCREEN**



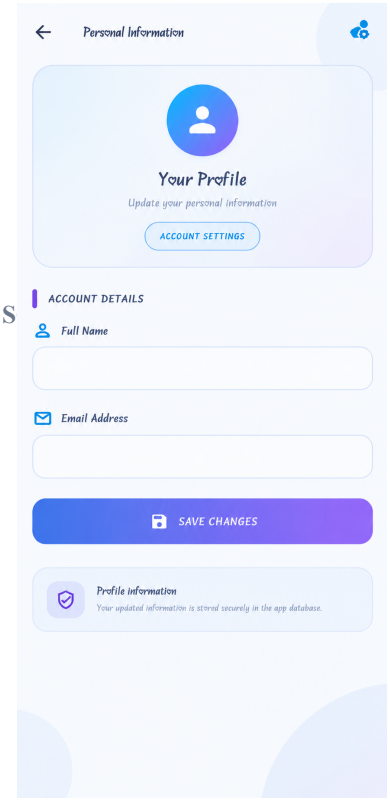

**ANALYTICS_SCREEN**



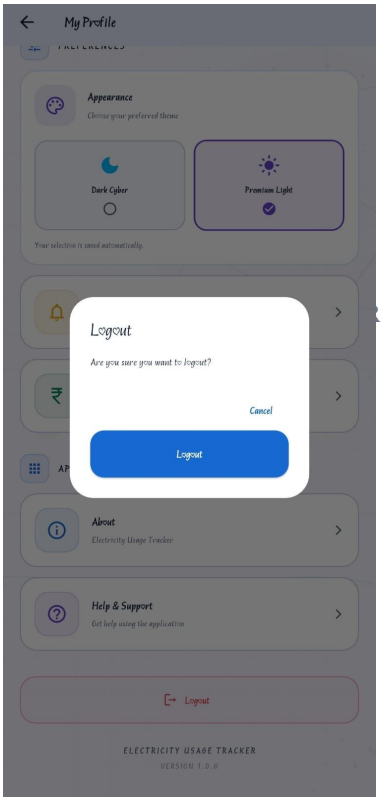

**ALERTS_SCREEN**




**PROFILE_SCREEN**




**PERSONAL INFORMATION_SCREEN**




**LOGOUT_SCREEN**

# 25. Security and Data Validation

The application stores account information and user-generated readings on the device. Security must therefore be considered at the interface, database, and device levels. The current implementation is suitable for a local academic project but is not production hardened.

## Application-Side Practices

- Validate required fields, email format, and password length.
- Reject negative meter values and decreasing readings.
- Confirm destructive actions before deleting data.
- Avoid showing raw database exceptions to the user.
- Use parameterised whereArgs queries rather than string concatenation.
- Clear the navigation stack on logout.

## Current Limitations

- Passwords are stored as plain text in the local users table.
- There is no password hashing, encryption, or secure session token.
- There is no account recovery or server-side authentication.
- A local database can be accessed on a rooted or compromised device.
- The app does not protect against a modified client because there is no backend authority.
- The app has no cloud backup or remote access control.

## Recommended Improvements

- Hash passwords with a slow, salted password-hashing algorithm.
- Encrypt sensitive local data with platform keystore support.
- Add a shared repository layer instead of duplicate database helpers.
- Add automated validation and security regression tests.
- Remove debug prints that expose internal data before release.

# 26. Conclusion

The Electricity Usage Tracker demonstrates how Flutter and Dart can be used to build a practical local mobile application. The project combines authentication, profile management, meter readings, bill calculation, usage history, monthly charts, alerts, theme selection, and SQLite persistence.

The architecture separates screens, models, database services, and theme code. This structure makes the application easier to understand and extend, while also revealing maintenance issues such as duplicate reading tables and plain-text password storage.

The project also demonstrates input validation, asynchronous programming, error handling, responsive layouts, reusable UI patterns, and custom-painted animation. These concepts provide a foundation for developing a more complete energy-management application.

## Final Project Summary

| Area | Implementation |
|---|---|
| Frontend | Flutter and Dart with Material 3 widgets. |
| Authentication | Local registration, login, and logout. |
| Profile | Name and email editing plus tariff configuration. |
| Readings | Add, list, delete, and clear meter readings. |
| Calculation | Units difference and tariff-based bill estimation. |
| Analytics | Average usage, average bill, highest usage, bar and line charts. |

## Final Project Summary (continued)

| Area | Implementation |
|---|---|
| Alerts | In-app high-usage warning, reminder, bill update, and energy tip. |
| Storage | sqflite databases for users, readings, and the tariff setting. |
| State | Local setState plus ThemeController for appearance. |
| Testing | Limited automated coverage; default widget test needs replacement. |
| Security | Local validation only; passwords are currently stored in plain text. |

This documentation focuses exclusively on the Electricity Usage Tracker implementation. The original SkillX material has been replaced with this project's actual authentication, reading, calculation, analytics, alert, profile, SQLite, testing, and security behaviour.

The report also identifies clear next steps: consolidate the duplicated data layer, repl ace the default
widget test, remove plain-text passwords, add real application screenshots, and cons ider a per-user
tariff if the application is extended beyond a single-device academic prototype.